Segment-02

17/03/2026
(Microprocessor)

★ 8086 Microprocessor architecture:

logically divided into two processing unit

Ⓐ Bus Interface unit (BIU)

- → Fetches instruction from memory
- → Reads data from memory an I/O ports
- → Writes data to memory and I/O ports

Contains:
- Segment Registers
- Instruction pointer (IP)
- Instruction queue

Ⓑ Execution unit (EU)

- → Recieves instruction codes and data from BIU
- → Executes instruction
- → Stores results in general registers or output through BIU

Contains:
- ALU
- General purpose register
- Index register
- Pointer
- Flag register

★ Register:

A small, faster data holding place inside the processor

2 types

General purpose/ Multipurpose
- → holds various data
- → used for almost any purpose

Special purpose registers
- → used for specific dedicated functions

CamScanner

★ Segment Registers

→ generate memory addresses when combined with other registers

→ each segment is a section of memory

Register	Full Name	Pupose
CS	Code segment	Holds program and praedures
DS	Data segment	Holds most data by a program
ES	Extra segment	Additional data segment (used in string instruction)
~~SS~~ SS	Stack segment	Defines the stack memory area

Detailed Description:

① Code segment (CS)

- → Holds the code (programs & procedures)
- → defines the starting address of the code section
- → Max size : 64 KB

② Data segment (DS)

- → Contains most data used by a program
- → defines the starting address of data section
- → Data accessed via offset address
- → Max size : 64 KB

③ Extra segment (ES)

- → Additional data segment used by string instructions
- → Holds destination data for string operations
- → Can be used as extra space for code or data
- → Max size : 64 KB

CamScanner

④ Stack Segment (SS)

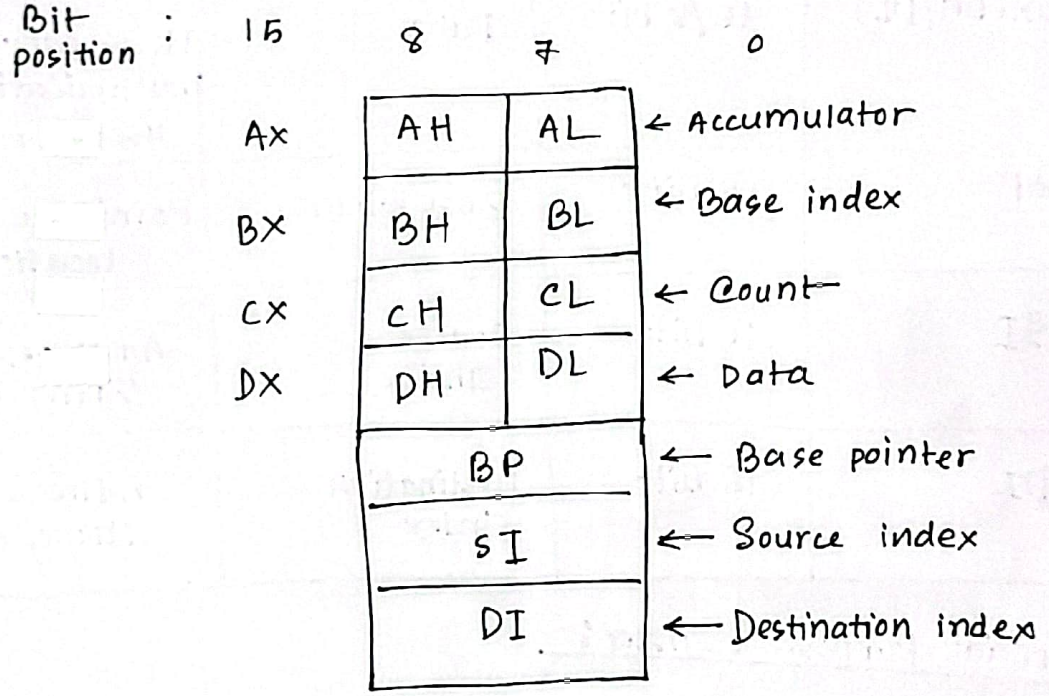
→ Defines the area of memory used for stack

→ Stack entry point determined by SS + stack pointer (SP)

★ General purpose register:

→ 8086 has 7 general purpose registers each in size 16 bits

→ Some can be split into two 8-bit halves



Detailed Description:

Register	Size	Name	Purpose
AX (AH/AL)	16/8 bit	Accumulator	Used in multiplication & division
BX (BH/BL)	16/8 bit	Base index	Holds offset addresses
CX (CH/CL)	16/8 bit	Count	Holds count for SHIFT, ROTATE, LOOP
DX (DH/DL)	16/8 bit	Data	Holds part of multiplication result; used for I/O
BP	16 bit	Base Pointer	Points to a memory location
SI	16 bit	Source Index	Addresses source string data
DI	16 bit	Destination Index	Addresses destination string data

* Special Purpose Register:

① IP → Instruction Pointer (16-bit)

→ Points to the next instruction in program
→ works with CS register

② SP → Stack Pointer (16-bit)

→ Addresses the stack memory area
→ Data stored / retrieved through this pointer

CamScanner

* Flag register (16-bit)

has 9 active flags:

- 6 status flag → indicate result conditions
- 3 control flag → control processor operations

Status Flags:

① Carry Flag (C)

→ hold carry after addition or borrow after substraction

```
  FFH     1111   1111
+ FFH     1111   1111
-------------------------
  FEH   (1) 1111   1110
```

(1) → carry out, so C=1

② Parity Flag (P)

→ counts the number of 1s in the result
→ Even number of 1s → P=1 (Even parity)
→ Odd number of 1s → P=0 (Odd parity)

FFH + FFH = 1111 1110
Number of 1s = 7 (odd) → P = 0

③ Auxilliary carry Flag (A)

→ Holds carry between bit positions 3 and 4 (half carry)
→ Used for BCD arithmatic

```
          4th ←   → 3rd
  FFH     1111   1111
+ FFH     1111   1111
-------------------------
        1 1111   1110
```

So, A = 1
otherwise A=0

④ zero Flag (Z):

- Z = 1, if the result of an operation is zero
- Z = 0, if the result is non-zero

⑤ Sign Flag (S)

- Reflects the arithmatic sign of the result
- S = 1 → Result negetive (leftmost bit = 1)
- S = 0 → Result positive

⑥ Overflow Flag (O)

- Indicates result exceeds the machines capacity
- Occurs when signed numbers are added / substracted
- O = 1 → overflow occured
- O = 0 → No overflow

Control Flags:

⑦ Trap / Trace Flag (T):

- Enables on chip debugging
- T = 1 → Debugging enabled
- T = 0 → Debugging disabled

⑧ Inturrupt Flag (I):

- Controls the INTR (Interrupt Request) input pin
- I = 1 → INTR pin enabled
- I = 0 → INTR pin disabled

⑨ Direction Flag (D):

Controls direction of string operations on SI / DI registers

D = 1 → Registers auto-decremented — reverse direction (STD instruction) ; apple

D = 0 → Registers auto incremented — forward direction (CLD instruction) ; elppa

CamScanner

★ Real Mode Memory Addressing:

- In real mode, the 8086 can address only the 1st 1 MB of memory
- Also called real memory or conventional memory
- Required by DOS operating system
- Windows does not use real mode

Segment and offset Address:

→ Every real mode memory address =

Segment Address + **offset Address**

Segment Address
- → Stored in segment register
- → defines start of 64 KB

offset Address
- → Selects specific location within 64 KB
- → Also called logical / displacement address

Physical address Calculation:

Physical Address = Segment Address × 10H + offset Address

Ex: Segment = 1000H, offset = 2000H

$$1000H \times 10H = 10000H$$
$$+\ 2000H$$
$$12000H \leftarrow \text{Physical Address}$$

Segment Size:

→ Each segment = 64 KB

→ If segment register = 3000H

First address = 30000H

Last address = 30000H + FFFFH = 3FFFFH

Default Segment : Offset Register Combinations:

Segment	Offset	Combination	Purpose
CS	IP	CS:IP	Code/ Instruction Address
DS	BX, SI, DI	DS:BX/SI	Data Address
ES	DI	ES:DI	String Destination Address
SS	SP or BP	SS:SP	Stack Address

Ex: CS = 1400H , IP = 1200H

→ Physical Address = 14000H + 1200H

= 15200H

Addressing modes of 8086

Address modes = the different ways ~~of~~ a processor can access data

Three Catagories:

① Data Addressing Modes

② Program memory Addressing Modes

③ Stack Memory Addressing Modes.

MOV AX, BX

Destination Source

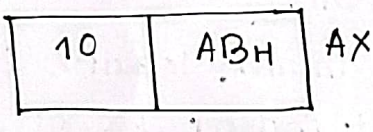
the contents of source register(BX) into destination register (AX)

① Immediate Addressing Mode:

- The data is part of instruction itself (appears directly)
- Data follows the opcode in successive bytes

Allowed:

MOV AX, 10ABH



Not Allowed:

→ MOV AH, 1000H
MOV 75H, AL

→ MOV CS, 1000H
(Segment register does not allow immediate data)

② Direct Addressing Mode:

- Moves a byte or word between a memory location and a register
- Memory address is directly specified in the instruction

Allowed:

MOV AX, [2004H] ⟶ Copies word at memory address 2004H into AX

MOV BX, LIST ⟶ Copies word at address LIST into BX

Not Allowed:

MOV [AX], [BX] ⟶ memory-to-memory transfer not possible

③ Register Addressing Mode:

- Data is stored in a register (8-bit or 16-bit)
- All registers except IP may be used.

ALLOWED:

MOV AX, BX ⟶ copies BX into AX (Both 16-bit)

MOV AL, BL ⟶ copies BL into AL (Both 8 bit)

CamScanner

Not allowed:

MOV AX, AL ⟶ different sizes (16 bits ≠ 8 bits)

MOV CS, AX ⟶ CS cannot be destination

MOV CS, DS ⟶ Segment to segment transfer not possible.

④ Register Indirect Addressing Mode:

- The address of the memory location is held indirectly in an offset register (BX, SI, DI)

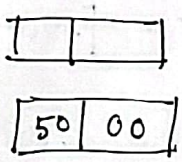
Allowed:

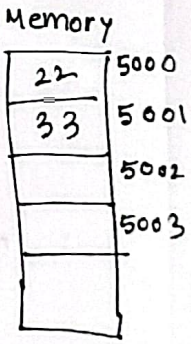
MOV AX, [BX] ⟶ BX holds the address; AX gets the data at that address

⑤ Indexed Addressing Mode:

- Offset of the operand is stored in an index register (SI or DI)
- Default segment: DS for SI and DI

MOV AX, [SI]



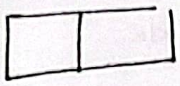


⑥ Register Relative Addressing Mode:

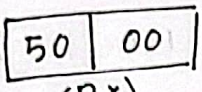
- Effective address = register (BX/BP/SI/DI) + displacement (8 or 16 bit)

MOV AX, [BX + 50H] ⟵ Effective address = BX + 50H

Ex: BX = 5000H
Displacement = 50H
Effective Address = 5000H + 50H
= 5050H
AX ⟵ memory [5050H]



AX



(BX)

+ 50H = 5050H

(offset) (Final Index Address)

⑦ Base Plus index addressing mode:

- Effective address = Base Register (BX or BP) + Index Register (SI or DI)

- MOV AX, [BX + SI]
- MOV AX [BX][SI]

} Same

Example:

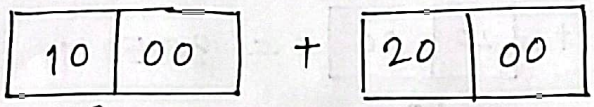
BX = 1000H

SI = 2000H

Effective address = 1000H + 2000H

= 3000H

AX ← memory [3000H]



(BX) (SI) = 3000H (Final index address)

Example:

MOV AX, [BX + SI]

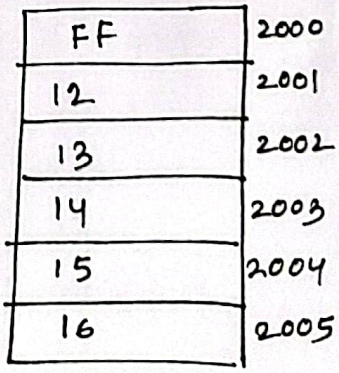
if BX = 2003H

SI = 001H

∴ Final index address 2004H

AX ← memory [2004H]

AX = 1615H



⑧ Base relative plus index addressing mode

- Effective address = Displacement + Base Register + Index Register

[Most complex addressing mode]

- MOV AX, 50H [BX][SI]
- MOV AX, [50H + BX + SI]

} Both are same

Example:

BX = 1000H
SI = 2000H
Displacement = 50H

Effective Address = 50H + 1000H + 2000H
= 3050H

AX ← memory [3050H]

50H + [10 | 00] + [20 | 00] = 3050H

Displacement — BX — SI — Final Index Address

C241424